
A Double Dueling DQN Agent for Highway Driving: Rule-Based Safety Filtering and a Crash-Level Diagnosis

Giuseppe D’Auria¹

Abstract

I implement a Double DQN agent with a Dueling architecture for highway driving among fifty vehicles and compare it against a defensive rule-based heuristic and a human driver. On held-out seeds the agent reaches 28.96 ± 4.32 return at a 10.0% crash rate, against 26.47 and 18.0% for the heuristic, but paired tests show the learned policy alone does not improve safety significantly. Adding a rule-based safety filter does: crashes fall to 1.0% on a reserved seed block while return rises. A three-step filter ablation isolates discretionary lane changes, not speed, as the source of the residual collisions, and a crash-level diagnosis explains why: in every collision examined the striking vehicle was already visible in the observation, so the failures are of decision rather than of perception. A human baseline reaching 33.4 return bounds what this interface allows. Throughout I use a training, selection and test seed split, which I found necessary because small-sample evaluation on this task is misleading enough to produce an apparent 0% crash rate for a model that crashes on 10% of held-out episodes. Code: https://github.com/cocopops9/RL-based_autonomous_driving.

1. Problem and critical issues

The environment is `highway-v0` from Highway-Env (Leurent, 2018). The ego vehicle drives on a three lane highway among fifty vehicles governed by the Intelligent Driver Model; an episode lasts at most forty decision steps and terminates on collision. The observation is a 5×5 kinematics matrix (ego plus the four nearest vehicles, features presence, x , y , v_x , v_y , the rows of the other vehicles relative to the ego), which I flatten to a 25 dimensional vector. The five discrete actions are `LANE_LEFT`, `IDLE`, `LANE_RIGHT`, `FASTER`, `SLOWER`. The per step reward is a weighted sum of a high-speed term (coefficient 0.4) and a right-lane term (0.1) with a collision penalty (coefficient -1), which the environment then normalizes into $[0, 1]$; the coefficients above are the pre-normalization values. The global seed is fixed to 0.

Three properties of the task shaped every design decision, and I consider them its critical issues.

Partial observability, and how little of it actually binds.

The state holds only the four nearest vehicles, and the default kinematic observation sets `see_behind=False`, so traffic astern is invisible entirely. I expected this to dominate and it does not; I had asserted otherwise before measuring. Two shipped measurements. The road is not crowded at the scale that matters: on average 4.4 vehicles lie within 100 m ahead (maximum 6), so four slots capture nearly all traffic in front. And instrumenting every crash to identify the collision partner, in all six diagnosed the partner had been in the previous observation and lay 17 to 30 m directly *ahead* (`results/crash_diagnosis.json`). The agent is not ambushed by what it cannot see; it drives into what it can. The residual crashes are a failure of policy, of when to brake and when to commit to a gap, not of perception. The rear blind spot matters only for lane changes into a faster vehicle astern.

A degenerate safe policy. A constant `SLOWER` policy collects a return of 26.94 ± 4.18 at a 6.7% crash rate (3 of 45 test episodes), so a learning agent can collapse into timidity and still look acceptable. Two things follow. Almost all available return is reachable with no intelligence at all, compressing the band in which learning can show value; and braking

¹University of Padova, Italy. Correspondence to: Giuseppe D’Auria <giuseppe.dauria@studenti.unipd.it>.

is not safe either, because `DiscreteMetaAction` exposes target speeds [20, 25, 30], so a braking ego floors at 20 m/s and still closes on slower traffic faster than a four-vehicle observation warns. That second point matters later. The trivial policy runs on a subset of the agents seeds, so the honest comparison is paired rather than two-sample: on the 45 shared seeds it crashes where the network does not three times and the network crashes where it does not six times, McNemar $p=0.51$, while the network earns +1.65 more return per episode. Braking and learning are thus indistinguishable on safety here, which is an argument for the safety filters of Section 6 rather than against them: only the filtered configurations clearly improve on doing nothing clever at all.

Evaluation noise. Episode outcomes are nearly binary: for the automated policies a completed episode returns roughly 29 to 31 (an attentive human reaches 33 to 34), a crash episode far less. Small evaluation samples therefore produce wildly varying estimates, and in hindsight this third issue, not the learning algorithm, was the crux of the project (Section 4).

2. Heuristic baseline

The baseline is a deterministic rule evaluated on the raw 5×5 observation at every step. I describe it exhaustively so that it can be reproduced from this text alone.

Let e be the ego row and $v_1, \dots, v_4$ the other rows, each with relative coordinates (x_i, y_i) and relative longitudinal speed $v_{x,i}$; a row with presence below 0.5 is ignored. Empirically one lane width corresponds to $|y| \approx 0.33$ in normalized units. The rule uses five constants: same-lane band $Y_s=0.15$, danger distance $D_c=0.10$, comfort distance $D_m=0.25$, and a lane clearance window of 0.30 ahead and 0.15 behind; the adjacent right lane is the band $y \in (0.15, 0.55)$ and the left lane $y \in (-0.55, -0.15)$.

At each step I compute $d^* = \min\{x_i : |y_i| < Y_s, x_i > 0\}$, the gap to the closest same-lane vehicle ahead ($d^* = \infty$ if none). The action is chosen by three priority levels:

1. if $d^* < D_c$ (imminent collision): change to the right lane if its clearance window contains no visible vehicle, otherwise to the left lane under the same condition, otherwise SLOWER;
2. if $d^* < D_m$: SLOWER, a preemptive gap opening;
3. otherwise the road ahead is clear: merge right if the ego is not already in the rightmost lane and the right window is clear (collecting the right lane bonus), else FASTER.

The design is deliberately defensive. An earlier, more aggressive variant that attempted overtakes crashed in the ma-

jority of episodes precisely because of partial observability: a lane that looks clear in the observation may contain an unobserved vehicle. Restricting lane changes to wide verified windows and defaulting to braking brought the crash rate down to 18.0% from a much higher figure, which makes the baseline a fair, non trivial benchmark rather than a straw man. On the held-out test seeds ($n=100$, same seeds as the agent) it achieves 26.47 ± 6.95 mean return at a 18.0% crash rate; the large spread is the bimodality of episode outcomes noted above.

3. Learning algorithm

3.1. Choice and rationale

The state is a low-dimensional continuous vector and the action space is small and discrete; this is the textbook setting for value-based deep RL, so I implemented DQN (Mnih et al., 2015) in PyTorch with two standard refinements rather than adopting a policy-gradient method whose extra machinery this problem does not need.

Double DQN (van Hasselt et al., 2016) computes the bootstrap target as

$$y = r + \gamma Q_{\theta^-}(s', \arg \max_a Q_{\theta}(s', a)) (1 - \text{done}), \quad (1)$$

selecting the action with the online network θ and evaluating it with the target network θ^- . This decoupling reduces the maximization bias of vanilla DQN, relevant here because the collision penalty makes the return distribution heavy-tailed and overestimating risky actions is exactly the failure mode to avoid.

Dueling architecture (Wang et al., 2016) decomposes $Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$. In free flowing traffic IDLE and FASTER have nearly identical values; separating the state value from the action advantages lets the network learn that a situation is safe independently of the specific action, which speeds up early learning.

3.2. Implementation

Two fully-connected layers of 128 ReLU units feed a value stream ($128 \rightarrow 64 \rightarrow 1$) and an advantage stream ($128 \rightarrow 64 \rightarrow 5$), with orthogonal initialization. Training uses Adam at 5×10^{-4} , $\gamma=0.99$, Huber loss, gradient-norm clipping at 1.0, batch size 64, a uniform replay buffer of 5×10^4 transitions, a warmup of 1000 transitions, and a *hard* target copy every 400 gradient steps. I initially used Polyak averaging ($\tau=0.005$) and observed slow Q value drift with intermittent policy collapses; periodic hard copies removed the worst of these oscillations, consistent with soft updates letting transient errors of the online network leak continuously into the bootstrap target.

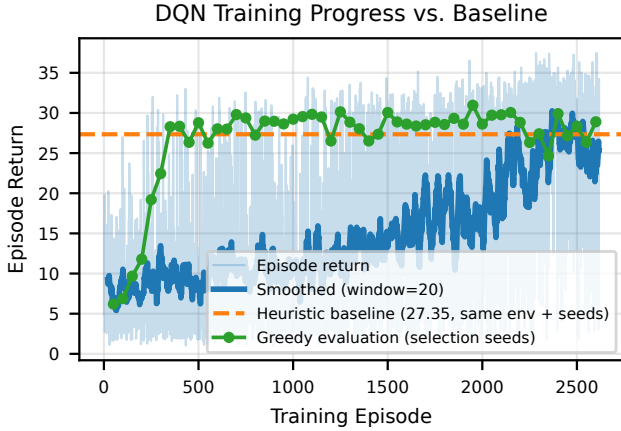


Figure 1. Learning against the baseline, over 2617 episodes (5×10^4 steps). Training returns include ϵ -greedy exploration and understate the policy, so the greedy series (green) is overlaid; the dashed line is the heuristic measured to match, same training environment (`ego_spacing=2.0`) and selection seeds.

Exploration is ϵ -greedy with a linear decay from 1.0 to 0.05. Because environment stepping dominates wall-clock time at this simulation frequency, I take two gradient updates per environment step, with the decay horizon and the target copy period expressed in gradient steps so that the schedule in environment steps is unchanged. Training runs 5×10^4 environment steps on `highway-v0` itself; an earlier attempt trained on the faster `highway-fast-v0` and evaluated on `highway-v0`, and the simulation frequency mismatch cost a substantial amount of crash rate, so I matched the two environments exactly. I did not preserve the logs of that comparison, so I report it as a design decision rather than as a measurement. The training loop checkpoints its complete state (network and target weights, optimizer, the full replay buffer, and the RNG states) with atomic rotated writes, so an interrupted cloud session resumes without losing learning progress. I should be precise about the limit of that claim: the environments own generator is not checkpointed and a resumed run reseeds the episode stream, so the resumption is faithful for the agent but not bit-identical for the environment. This proved necessary in practice on preemptible free-tier sessions.

4. Evaluation methodology

Episode outcomes on this task are close to binary, which makes small-sample evaluation unusually treacherous. This section sets out the protocol and the two measurements that motivated it.

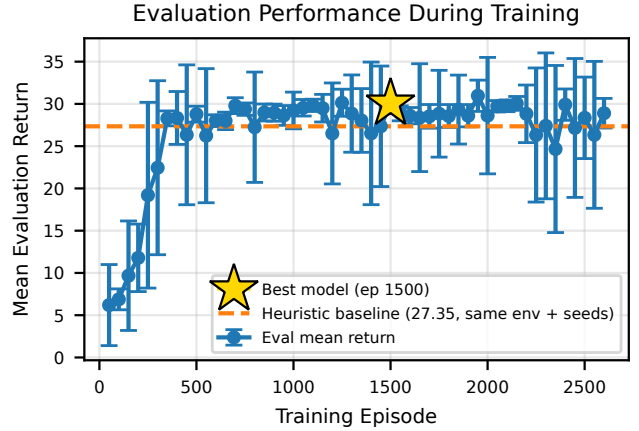


Figure 2. Greedy return on the fixed selection seeds, star marking the checkpoint chosen by the safety-weighted score. Ten episodes per point, so these values are systematically more optimistic than the held-out numbers of Table 1, which is the point of Section 4.

Selection on a noisy criterion is biased upward. I evaluate the greedy policy every fifty episodes on a fixed seed set and keep the checkpoint with the best safety-weighted score

$$S = \bar{R} - 10 \hat{c}, \quad (2)$$

where $\bar{R}$ is the mean return and $\hat{c}$ the crash rate; the weight encodes that one extra crash per ten episodes must cost more than one point of mean return. The selected checkpoint reports 30.06 return at 0% crashes. That figure is not an estimate of its performance. Taking the *argmax* of a noisy quantity over dozens of checkpoints and then reporting that maximum is upward biased, the winner’s curse: the periodic evaluations use ten episodes each, so for a policy with true crash probability $p=0.2$ the standard error of a single evaluation is $\sqrt{p(1-p)/10} \approx 0.13$; observing zero crashes once across fifty two such evaluations is expected even for a mediocre policy. The selection score is a valid *criterion for choosing* between checkpoints, but it is not a valid *estimate* of performance.

Protocol. I split the seeds three ways, exactly as one splits data. *Training* uses seed 0. *Selection* (seeds 12345+ i) is used only to choose the best checkpoint and to decide best versus last; that is what a validation set is for. *Test* (seeds 90000+ i , disjoint, one hundred episodes) is touched exactly once, on the finally chosen model, and never used for any decision. The number I report is the test number, and only that.

The size of the effect. The same weights evaluated on the test seeds give 28.96 return at 10.0% crashes. The gap between 0% and 10.0% is entirely a property of the protocol, not of the model, and it is large enough to invert a conclusion.

Table 1. Comparison on the TEST seed set (90000+ i), $n=100$ episodes per automated policy in one identical environment ($\text{ego_spacing}=1.5$), baseline included. The manual row is $n=10$ and the holdout row uses the untouched block 50000+ i ; neither is $n=100$. $S = \bar{R} - 10\hat{c}$ as in Eq. 2; SE is the binomial standard error of the crash rate, and $\pm$ on returns is the standard deviation across episodes, not a standard error.

	mean return	crash	SE	S
Manual control (human)	33.41 $\pm$ 0.50	0%	–	33.4
DQN + conservative	29.33 $\pm$ 1.57	2.0%	1.4%	29.1
DQN + strict	29.35 $\pm$ 2.77	6.0%	2.4%	28.8
DQN + smooth	29.01 $\pm$ 3.98	9.0%	2.9%	28.1
DQN raw, test	28.96 $\pm$ 4.32	10.0%	3.0%	28.0
Heuristic baseline	26.47 $\pm$ 6.95	18.0%	3.8%	24.7
DQN last iterate	26.32 $\pm$ 8.39	31.0%	4.6%	23.2
DQN, selection (biased)	30.06	0%	–	30.1
Conservative, holdout	29.13 $\pm$ 2.68	1.0%	1.0%	29.0

Zero crashes in N episodes bounds the true rate only below roughly $3/N$ at 95% confidence.

One environment for every policy. Baseline, training and evaluation all use the single configuration $\text{ego_spacing}=1.5$, and every number reported here is an evaluation in that environment. This matters because the parameter is not innocuous: measured at the environment default of 2.0, the same agent looks better than it is.

The selection criterion saturates. Twenty one of the fifty two periodic evaluations scored exactly zero crashes and ten checkpoints tie within one point of S : at ten episodes per evaluation the criterion cannot rank the candidates it was used to rank, and the choice among tied leaders is close to arbitrary. The late-training crash rates on the selection seeds oscillate across 0.0, 0.1, 0.3, 0.2, 0.6, 0.2, 0.4, 0.1, 0.7, 0.0, so it also overstates matters to say the training curve looked healthy while the final weights were poor; Figure 2 shows visible instability, and a reader watching it would have had fair warning. Selection rescued the run, but on a criterion too noisy to deserve the credit.

A fourth block for the filter choice. The three safety filters of Section 6 are compared on the 90000+ i seeds, which also supplied the crash diagnosis and the choice of which filter to ship. Those seeds therefore act as a selection set for the filter, so the 2% in Table 1 carries the same upward bias as any selected quantity. I evaluated the chosen filter, unchanged, on a fourth block (50000+ i , $n=100$) reserved for this single claim: it scores 29.13 ± 2.68 at 1.0% crashes. That is the figure I claim for the system.

An alternative explanation, tested and rejected. A referee might object that the selection and test blocks simply differ in difficulty, which would explain the gap without any winner’s curse. This is checkable: I ran the heuristic, a fixed policy that no part of this project tunes, on both blocks. It

crashes on 22.5% of the selection seeds (9 of 40) against 18.0% of the test seeds, a difference of 4.5 points at $z=0.61$, not significant and in the wrong direction for the objection. The selection block is, if anything, slightly harder, so the optimistic selection numbers are not explained by easy seeds and the winner’s curse account stands.

Training and evaluation spacing. The shipped weights predate this standardization and were trained at $\text{ego_spacing}=2.0$, while every evaluation uses 1.5. The direction works against the reported result, since the agent learned in the sparser setting and is scored in the denser one, so the 10.0% crash rate is conservative. Two consequences: `training.py` does not reproduce the shipped weights bit for bit, and the during-training curves were collected at 2.0, so their baseline reference line is measured the same way (27.35, $n=40$).

5. Results

Learning is measured as described in Section 4: periodic greedy evaluations on the fixed selection seeds during training (Figure 2), against the baseline as a horizontal reference (Figure 1), with the final claim made once on the held out test set (Table 1). A second, manually driven baseline (`manual_control.py`, keyboard control over ten episodes) is provided as required. Driven attentively it completes every episode at 33.41 ± 0.50 return and 0% crashes, above all automated policies. My own rule of three applies to me too: zero crashes in ten episodes bounds the human rate only below roughly 26%, so the ceiling is established on return, not on safety. Since it is played interactively rather than on the fixed test seeds, I treat it not as a seed matched comparison but as an empirical performance ceiling: no policy here, learned or scripted, comes close to it on return, and I treat it as the practical ceiling for this observation and action interface rather than as a proven optimum.

Three readings of Table 1. First, the selection row and the test row are the same weights measured on different seeds; the residual gap between them (0% versus 10.0% crashes) is the selection bias made concrete, and it is the honest row that counts. Second, the safety-weighted score S orders the policies cleanly: conservative (29.1), strict (28.8) and smooth (28.1) filters above the raw network (28.0), all above the baseline (24.7). The raw network improves on the baseline in mean return (28.96 versus 26.47) and crash rate (10% versus 18%). Because every policy runs on the identical seed set, the correct instruments are paired and more powerful than the two-proportion tests I first used: McNemars exact test on discordant episodes, and a paired t on returns. Against the baseline, the conservative filter turns 17 baseline crashes into 1 ($p=0.0001$), strict 16 into 4 ($p=0.012$), smooth 16 into 7 ($p=0.093$), raw 17 into 9 ($p=0.169$). Under Holm correction only the two stronger filters survive

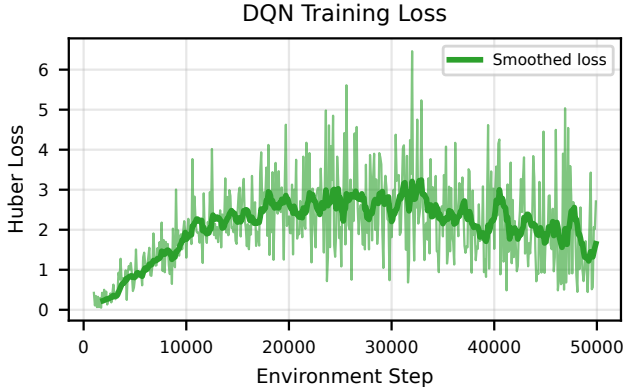


Figure 3. Huber loss during training (linear scale, sampled every 100 environment steps, i.e. every 200 gradient updates at the two updates per step ratio). The loss does not converge to zero and drifts upward as surviving episodes lengthen the bootstrapped targets; an increasing loss here does not indicate divergence, and it is why model selection uses the evaluation score rather than the loss.

at 5%: the learned policy alone does not demonstrably improve safety over the heuristic, the filtered configurations do. On return every configuration beats the baseline (paired t from 2.90 to 4.27, +2.5 to +2.9). The two filters are not separable (6 against 2 discordant, $p=0.29$). Per-episode outcomes ship in `results/episode-vectors.json`; `paired-tests.py` recomputes every statistic here from them, and `results/crash-rate.pdf` traces crashes over training. Third, checkpoint selection was decisive: the shipped checkpoint is a mid-training iterate at $\varepsilon \approx 0.53$, step 20,801 of 50,000, so the final three fifths of the budget were not merely wasted but harmful. The last iterate, tested on the same seeds, scores 23.2 at a 31% crash rate, below the baseline, so shipping the final weights would have produced a worse-than-baseline agent. I resist the gloss that this happened invisibly: Section 4 records that the periodic evaluations were openly unstable in the final third, so the warning was on the page. The narrower point stands: the Huber loss (Figure 3) cannot serve for selection, whereas the evaluation score at least tracks the optimized quantity.

6. Discussion

6.1. Why the agent wins, and by how much

The heuristic reacts to one scalar, the nearest same-lane gap, with a fixed geometric lane-change test; the network conditions on all four observed vehicles’ relative positions and velocities, so it can distinguish a slow car being approached quickly from one moving at the ego’s speed and time overtakes accordingly. On the test set this shows up where the extra information should matter: with the safety filter, fewer

crashes at higher speed and a much smaller return spread. The margin over the heuristic is honest but bounded. Two references frame it. Below, the always-brake policy reaches a return near 27 with no learning at all, so the exploitable band above a trivial policy is narrow. Above, attentive manual play reaches 33.4 return (Table 1), an empirical ceiling on return that no automated policy here approaches, though its 0% over ten episodes is too small a sample to bound the human crash rate tightly; the agent and all three filters sit between these two references on return, closer to the human than to the trivial policy. On crashes the picture is less flattering and worth stating plainly: the raw network at 10% is no better than the constant-braking reference at 6.7%, so the learned policy buys its extra return without buying safety, and only the filtered configurations improve on both axes at once. With near-binary episode outcomes, the crash rate standard error at 100 episodes is about three points for a single policy, enough, on the paired tests of Section 5, to resolve the two stronger filters from the baseline while the gap between them remains within noise ($p=0.29$).

6.2. Driving the crash rate down with a safety filter

To reduce collisions further I instrumented the agent’s behavior and post-processed its actions with a small deterministic filter, keeping the network as the primary policy. The action histogram of the raw greedy policy, measured over 444 steps and shipped as `results/action-histogram.json`, shows lane changes on 37.4% of steps (4.7% left, 32.7% right) at a mean normalized speed of 0.267. To separate the two candidate causes I probed the speed cap on its own, adding it to the strict filter without the lane-change restriction (CappedStrictFilter). Over eighteen test episodes it crashes twice, 11.1% with a Wilson interval of [3.1, 32.8]%, against one crash for the strict filter on the same eighteen seeds. The probe is underpowered: it fails to show a benefit from the cap and cannot exclude one. The weight rests on the paired comparison at $n=100$, where the conservative filter, adding cap *and* lane-change restriction, converts six strict crashes into two. The lane-change restriction is the attribution I defend: a lane that looks clear in the four-vehicle observation can be entered into an unobserved car. I therefore built the filter in three increments of rising conservatism, so that the contribution of each rule is visible. The chain is not a clean ablation and I flag that rather than overclaim it: `StrictFilter` adds pre-braking but also retunes the same-lane band (0.12 to 0.13) and the braking distance (0.09 to 0.13), so the smooth to strict step moves two things at once, and the strict to conservative step bundles the speed cap with the lane-change restriction. The *smooth* filter applies only a lane-change cooldown and emergency braking; it gives 29.01 ± 3.98 return at 9% crashes, barely better than the raw network, which shows that suppressing merely the rapid oscillation between lanes is not what helps.

The *strict* filter adds pre-braking near slow traffic; over one hundred test episodes it gives 29.35 ± 2.77 return at 6% crashes. The *conservative* filter additionally caps speed and restricts lane changes to genuine emergencies (a same-lane vehicle inside the braking distance); it reaches 29.33 ± 1.57 return at 2% crashes (2 of 100) with a much tighter return spread. The ablation is monotone in crash rate, 10%, 9%, 6%, 2%, and the largest single drop comes from the last increment, the one that restricts discretionary lane changes. All three filters improve return and safety simultaneously over the raw network. At $n=40$ the two were indistinguishable and I framed them as points on a speed-safety frontier; at $n=100$ that fails. Conservative dominates strict, matching its return (29.33 versus 29.35, inside noise) at a third of the crash rate, so no dimension favours strict and conservative is what I would ship. I report all three because the progression is what identified lane changes, rather than speed, as the source of the residual crashes: the smooth row is the control that rules out oscillation damping, and the conservative row supplies the effect.

6.3. The largest uncontrolled variance is the one I did not control

This paper argues that evaluation variance dominates, and controls it carefully: four disjoint seed blocks, a hundred episodes per row, standard errors throughout. It then rests every conclusion on *one* training run. Training seed is routinely a larger source of spread than evaluation seed (Henderson et al., 2018; Agarwal et al., 2021), and I did not sample it. The honest scope is therefore narrower than the phrasing suggests: "the agent beats the baseline" means *this* run, selected by *this* criterion, does so on held-out seeds. Whether the procedure does in expectation is untested, and more seeds would be my first additional experiment. A paper making my argument has no business leaving its largest variance term unmentioned. No hyperparameter search was performed: the values in Section 3 were fixed from the literature and two documented corrections, so nothing here was tuned on any seed block.

6.4. Possible improvements

In order of expected value, revised by the crash diagnosis: since every diagnosed collision was with an already-visible vehicle, enlarging the observation is *not* the first lever. I would attack the decision instead, with a risk-sensitive objective that prices committing to a gap against the lost braking option; then more training seeds; then a recurrent encoder (Hausknecht & Stone, 2015) for the rear blind spot; prioritized experience replay (Schaul et al., 2016) to revisit the rare pre-crash transitions more often; and evaluating on several hundred test episodes to tighten the crash rate confidence interval. I would attempt the observation change and the larger test set first.

7. Conclusion

A Double Dueling DQN agent trained on `highway-v0` outperforms a carefully specified defensive heuristic on return, but the paired tests show it does not significantly improve safety on its own: at 10% crashes it is indistinguishable from a policy that simply brakes. What closes that gap is a small rule-based filter over the networks actions, which brings crashes to 1.0% on a reserved seed block while raising return, and the three-step ablation locates the effect in the restriction of discretionary lane changes rather than in the speed cap. A crash-level diagnosis explains the mechanism: in every collision examined the striking vehicle was already present in the observation, so the residual failures are of decision, not perception, and enlarging the observation would not be the first thing to try. Two limitations bound all of this. The conclusions rest on a single training run, which is the largest variance term left uncontrolled; and, in a task whose episode outcomes are near-binary, the evaluation protocol determines both which model one ships and what may be claimed for it, a point sharp enough here that a ten-episode measurement of the shipped model reads 0% where a hundred-episode one reads 10%.

References

- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., and Bellemare, M. G. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.
- Hausknecht, M. and Stone, P. Deep recurrent q-learning for partially observable mdps. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 3207–3214, 2018.
- Leurent, E. An environment for autonomous driving decision-making. <https://github.com/eleurent/highway-env>, 2018.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., and Ostrovski, G. Human-level control through deep reinforcement learning. *Nature*, 518: 529–533, 2015.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.
- van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *Proceedings of*

the AAAI Conference on Artificial Intelligence, pp. 2094–2100, 2016.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pp. 1995–2003, 2016.